GIT

Why make the effort to learn git?

1. Will save you lots of time in the future
2. Will allow you to collaborate with various people, simultaneous coding
3. Will give you a history of you and your classmates' work
4. Will *force* you to write clean and understandable code
5. Will make sure you don't lose your work :)

What is the basic concept?

- One repository storing the code base
- Collaborators can *clone* this repo on their local computer (origin vs. local - original repo vs. local copy)
- You can create *local* branches of the code to work on new features, make changes and merge them with the *original* code base
- Origin repo is the source of truth, you need to *locally* adapt to make sure you are on track with the original code and do not overwrite any changes
  (pull changes, resolve conflicts, etc.)

Some rules…

- Always pull from the original repository before starting coding
  → You make sure you are up to date with the current code base
- Use commit messages that are understandable to your peers
  → Other people will know why you made certain changes
  → Will help you to remember your thinking process
- When resolving conflicts (>1 collaborator changes the same line of code) make sure you do not overwrite important changes
  → Communicate with your team member: Why did you both change the same code?
  → Organize your work beforehand to avoid these kind of situations

Let’s put this into practice

Please get together in your groups and open the slides on your Git class

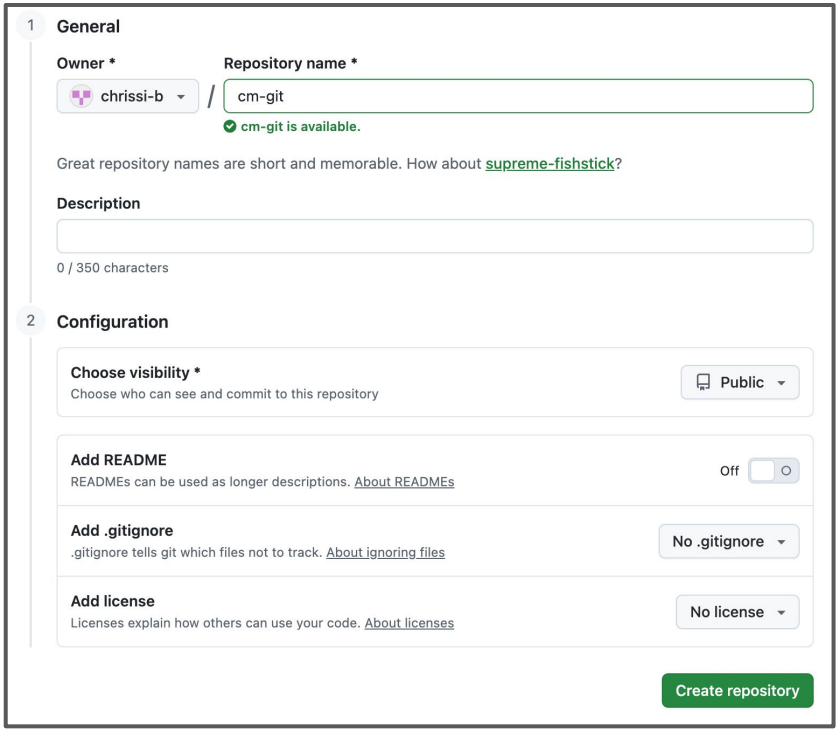
We will simulate a collaboration of two team members

Team Member 1 **Team Member 2**

Team Member 1

Create a repository

- Sign in to GitHub
- Go to *Repositories* (click on your profile picture on the top right to open menu)
- Click on **`New`**
- Choose your repository name and click on **`Create repository`**

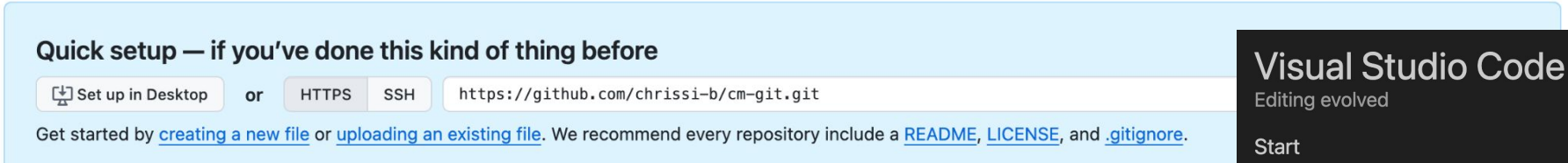


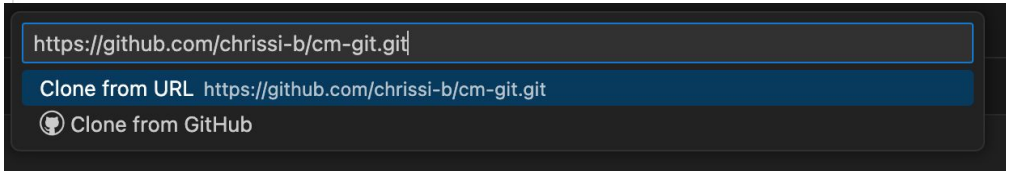
Clone the repository from your computer

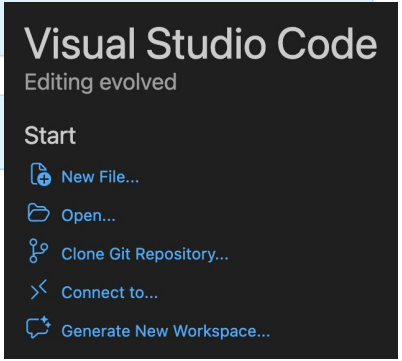
Team Member 1

Team Member 2

- Open an IDE (Integrated development environment) of your choice: Visual Studio Code, IntelliJ, etc.
- Find the option *Clone Git Repository*
- Use the repository link provided by GitHub
- Choose a destination on your computer and confirm





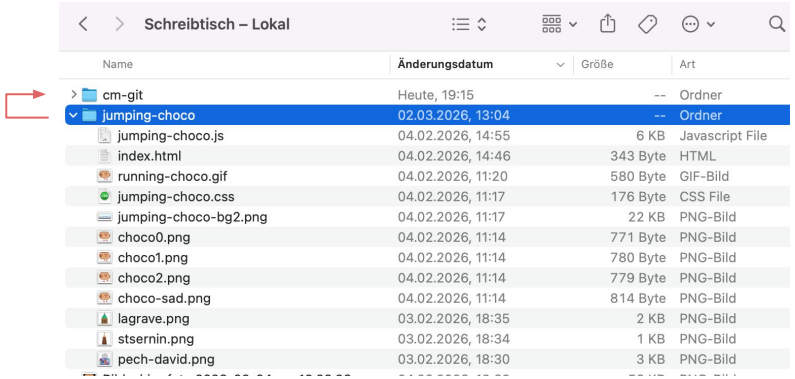


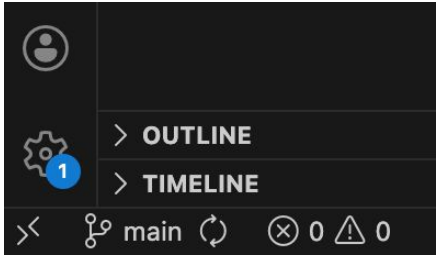
Team Member 1

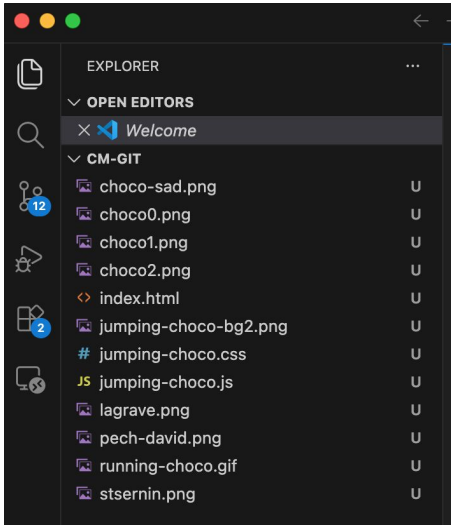
Add your project code to the folder

- You are now on the main branch
- Take the code you have written so far and add it to your repository folder

→ find destination folder in your file explorer
→ simply copy paste your files



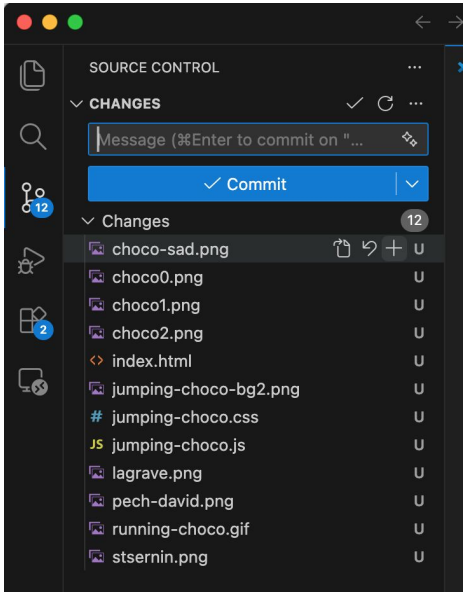




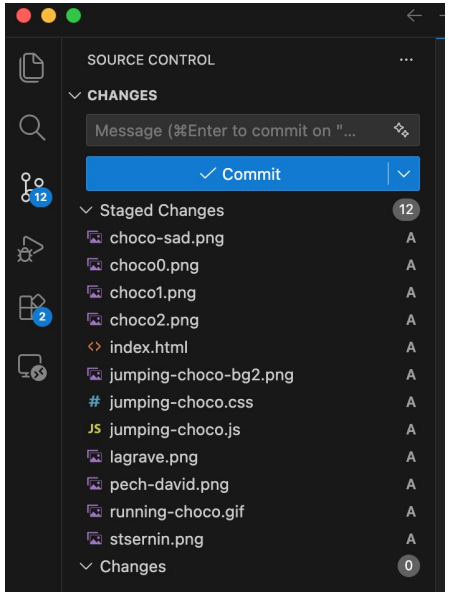
Team Member 1

Stage all changes

- staging = choose the files with changes that you want to include in your next commit (basically the changes you want to confirm and push to the original code base)
- stage all changes (we want all files to be in the original code base)
- click on **+** for all files



before

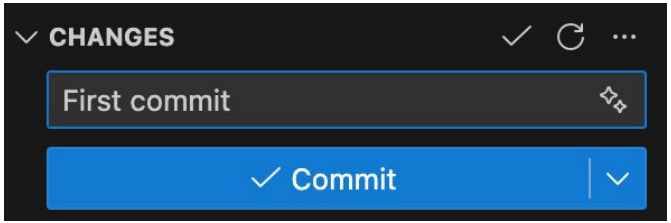


after

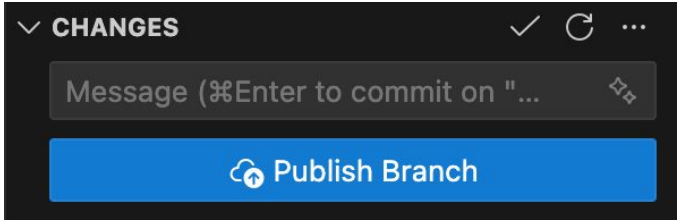
Team Member 1

Make your first commit and push changes

- write your commit message
- commit

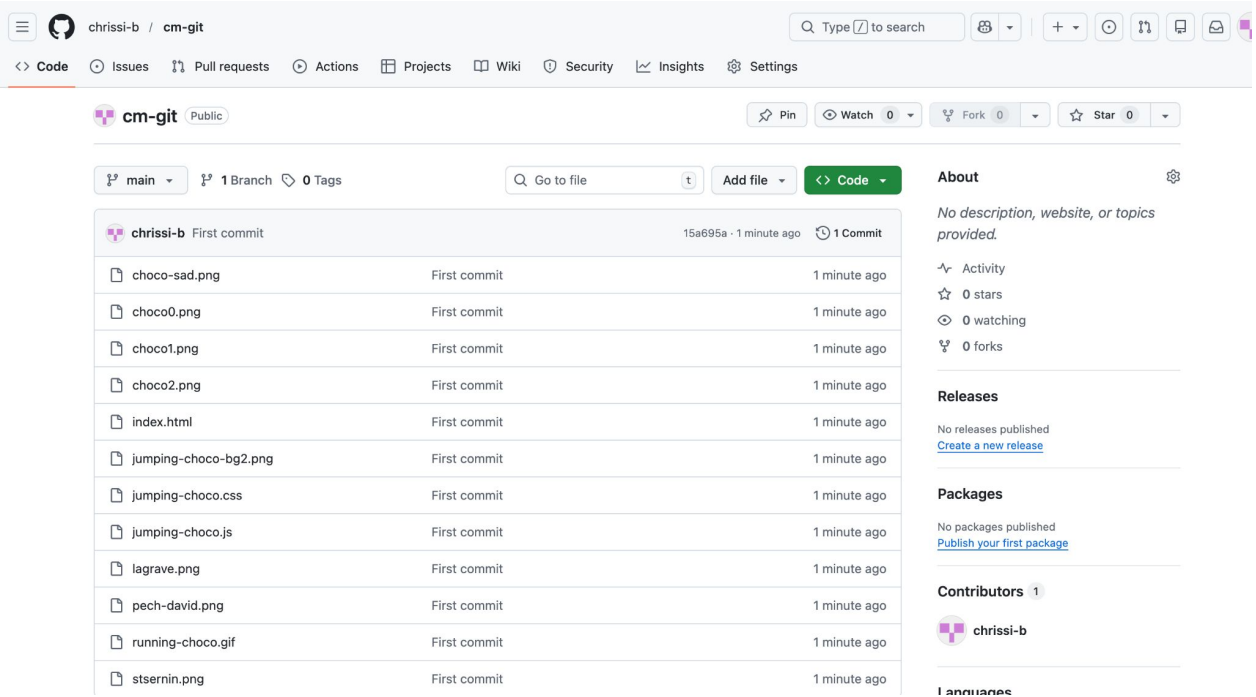


- publish branch to push to the remote repository



Team Member 1

Go to GitHub and check if commit was successful



Team Member 2

Pull changes

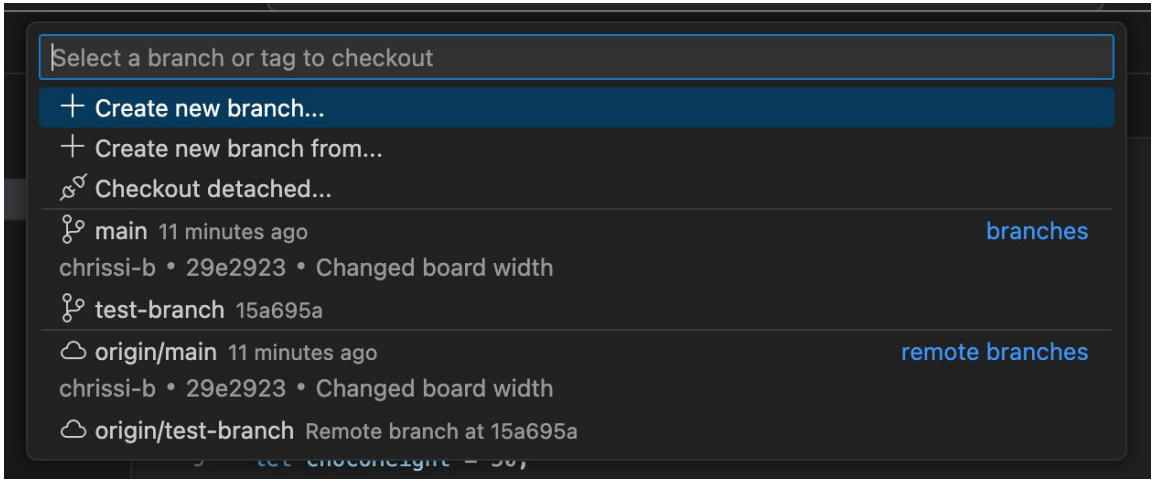
- On your computer, none of this will be visible yet
  → Remember you are on a local copy of the original repository
  → You actively have to **pull** changes to the original repository on your local copy
- Open a terminal in your IDE and execute **git pull**
- This will pull the original repo and update your local copy

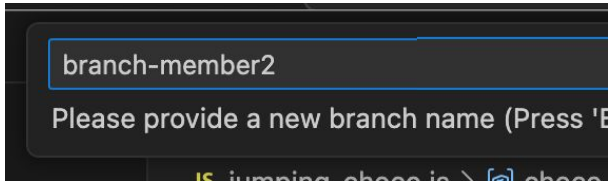
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(base) christinaberghegger@MacBook-Pro-von-Christina cm-git % git pull
Already up to date.
(base) christinaberghegger@MacBook-Pro-von-Christina cm-git %
```

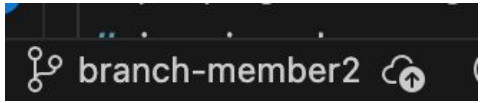
Team Member 2

Create a new branch

- We will simulate that you work on a different feature, for which you create a new feature brand
- Click on **main** and choose **+ Create new branch**







Make changes to the code

- We will simulate conflicting code → Team member 1 commits a change to main which concerns the same code that Team member 2 will change in their own branch
- This will require conflict handling when merging feature and main branch

Team Member 1

Team Member 2

```
//board
let board;
let boardWidth = 700;
let boardHeight = 320;
let context;
```

Team Member 1

in main

```
jumping-choco.js ×
jumping-choco.js > ...
1  //board
2  let board;
3  let boardWidth = 500;
4  let boardHeight = 320;
5  let context;
```

```
//board
let board;
let boardWidth = 400;
let boardHeight = 320;
let context;
```

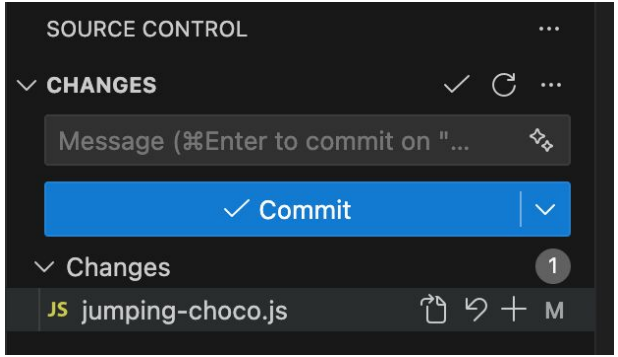
Team Member 2

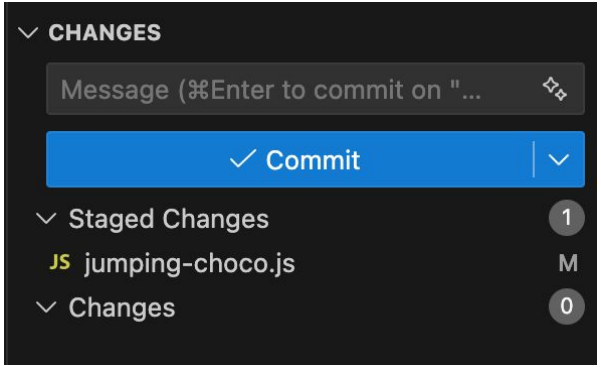
in branch-member2

Team Member 1

Stage, commit, and push changes

- Only **Team Member 1** will stage changes **main**



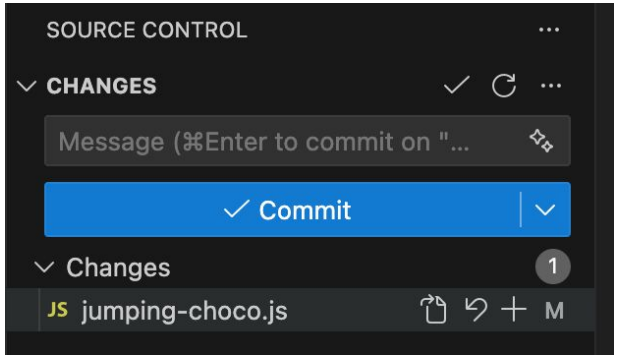


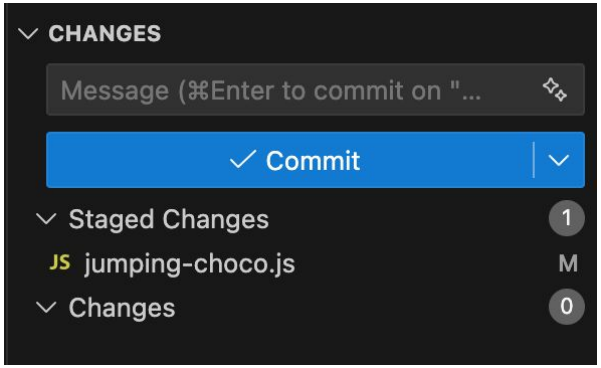
- Then write your commit message, commit and publish your branch (push)
- Now the main branch in the remote repository contains a commit that is not included in the branch branch-member2 of **Team Member 2**

Team Member 2

Stage & commit changes

- Only **Team Member 2** will stage changes





- Then write your commit message and commit
- **DO NOT PUSH YET**

Team Member 2

Fetch main and merge

- When working on a feature branch other than main, the goal is to always make sure there are no conflicts before pushing commits
- Therefore, we **fetch** main (gets all commits that have been performed on the original repository without updating your own local code/branch) and **merge the origin/main branch** into your local branch
- This forces you to solve any conflicting code with the original repository and makes sure your changes are compatible with the main branch

```
● (base) christinaberghegger@MacBook-Pro-von-Christina cm-git % git fetch
```

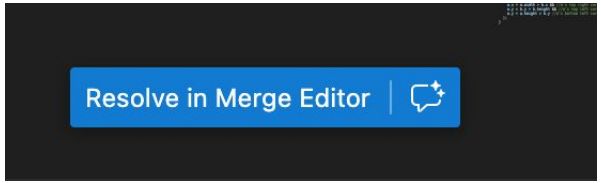
```
(base) christinaberghegger@MacBook-Pro-von-Christina cm-git % git merge origin/main
Auto-merging jumping-choco.js
CONFLICT (content): Merge conflict in jumping-choco.js
Automatic merge failed; fix conflicts and then commit the result.
```

Team Member 2

Resolve conflicts

- You will see the conflicting code lines indicating your local code (HEAD) and the code in the remote repository (origin/main)
- Depending on your IDE, you can easily resolve conflicts in a Merge Editor

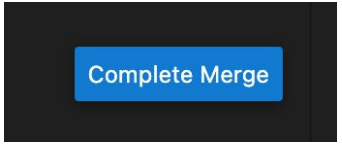
```
JS jumping-choco.js > ...
1   //board
2   let board;
    Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
3   <<<<<<< HEAD (Current Change)
4   let boardWidth = 400;
5   =======
6   let boardWidth = 700;
7   >>>>>>> origin/main (Incoming Change)
8   let boardHeight = 320;
9   let context;
10
11  let gameOver = false;
12  let score = 0;
13
```

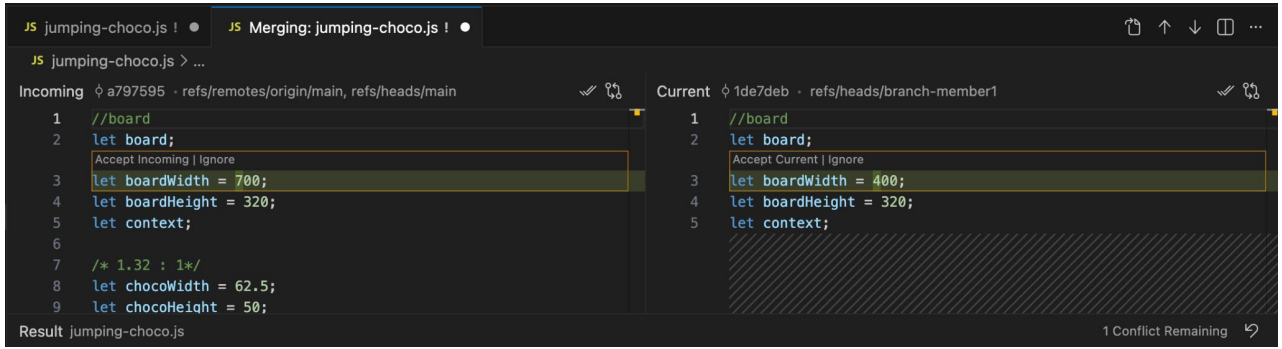


Team Member 2

Resolve conflicts

- Decide which one is the correct
- Accept Incoming (left) will take the code from origin/main
  Accept Current (right) will keep your current code
- Complete merge
- Make sure your code runs as expected after the merge
- After the merge you can safely stage, commit and push your changes to your remote branch

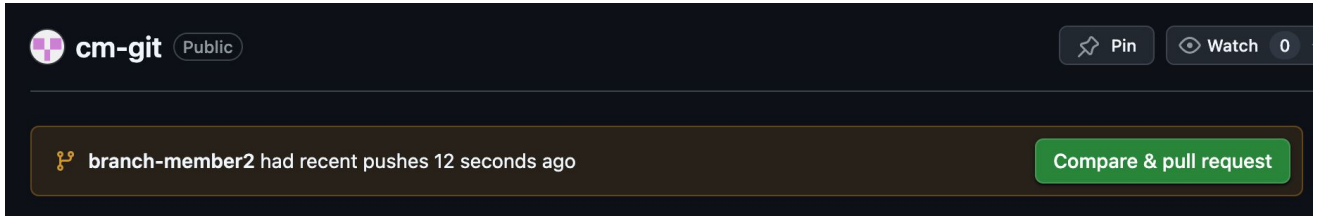




Team Member 2

Create a merge request on GitHub

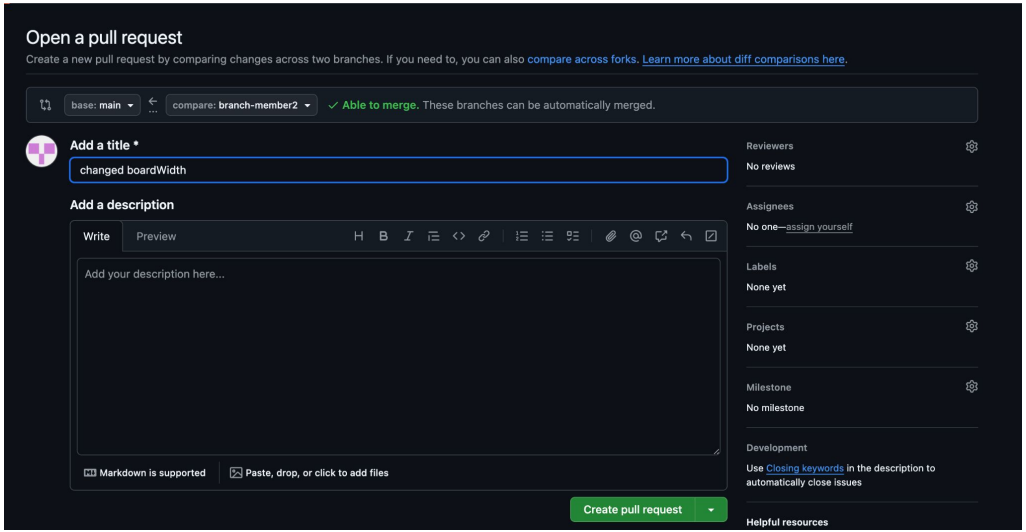
- When pushing to a branch other than main, GitHub usually suggests to pull the changes into main

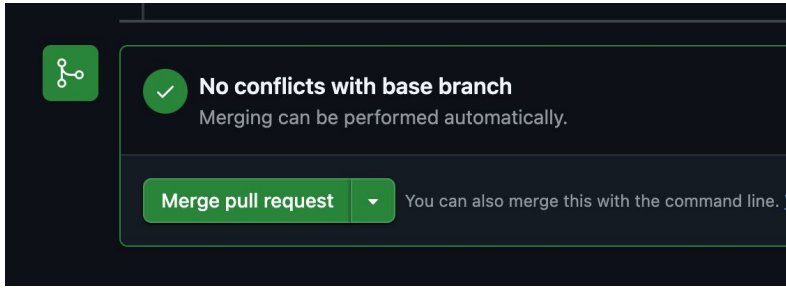


Team Member 2

Create a pull request on GitHub

- The Able to merge notification shows us that there are no conflicts with main and we can safely merge both branches (we accept the changes Team Member 2 implemented)





What we did

- Two team members worked on the same code simultaneously
  → While Team Member 2 worked on a different feature in their own branch *branch-member2,* updates happened to the main branch
- We resolved conflicts between feature branch *branch-member2* and the main and could then confidently merge the two branches